

Simple Design Pattern Breakpoint Demo

Quick reference first. Short live script starts on page 2 and begins with Facade.

1. Facade Pattern

Breakpoint location

app/services/separation_service.py, line 135:
`uploaded_audio = get_uploaded_audio(db, audio_id)`

How to trigger

Select a WAV file and click Run separation. Upload runs first, then separation reaches this Facade method.

What to say

Now I will show the Facade. As you can see, execution stops inside SeparationService. When I step forward, this Facade calls subsystem services such as ModelService, StorageService, and ResultService.

2. Factory Method

Breakpoint location

app/routers/upload.py, line 28: `validator = audio_validator_factory.create_validator(file.filename)`

How to trigger

Select a WAV file and click Run separation. The first backend action is upload validation.

What to say

Now I will show Factory Method. As you can see, UploadRouter asks the factory for a validator instead of creating one directly. Step in to show WavAudioValidatorFactory returns WavAudioValidator.

3. Strategy Pattern

Breakpoint location

app/ml/separation_engine.py, line 29: `return self.algorithm.separate(...)`

How to trigger

Click Run separation and continue until the separation workflow reaches the ML engine.

What to say

Now I will show Strategy. As you can see, SeparationEngine calls `self.algorithm.separate`. When I step into it, the real object is NeoSSNetStrategy.

Demo order: Facade first, then Factory Method, then Strategy, then DBeaver history query.

Short 5-Minute Script

Say this during the video. It is action-focused and starts with Facade.

0:00-0:20 - Demo goal

Now I will show how the design patterns work in the real backend code. I will start with Facade, then show Factory Method, then Strategy. After that, I will switch to DBeaver and show that SQLite records the separation history.

0:20-1:35 - Facade Pattern

Now I will click Run separation and show the Facade pattern. As you can see, the breakpoint stops at `separation_service.py` line 135, inside `SeparationService`. This class is the Facade. The route does not control all the separation steps directly. When I step forward, `SeparationService` calls `get_uploaded_audio`, then `ModelService` at line 136, `StorageService` at lines 140 and 156, and `ResultService` at line 169 after inference. So the subsystems do the detailed work, while the router only calls one simple service method.

1:35-2:30 - Factory Method Pattern

Now I will show Factory Method using upload validation. I click Run separation with a WAV file. As you can see, the breakpoint stops at `upload.py` line 28. `UploadRouter` is the client. It does not directly create `WavAudioValidator`. It asks `AudioValidatorFactory` for a validator. When I step into the call, it goes to `WavAudioValidatorFactory` at line 46, and line 49 returns `WavAudioValidator`. This means object creation is separated from the upload route.

2:30-3:35 - Strategy Pattern

Now I will continue to the Strategy pattern. As you can see, execution reaches `SeparationEngine`. At line 29, the engine calls `self.algorithm.separate`. `SeparationEngine` is the Context, and `SeparationAlgorithm` is the interface. When I step into the call, it goes to `NeoSSNetStrategy`. This proves the engine uses an algorithm strategy instead of depending directly on `NeoSSNet` internals.

Short 5-Minute Script

Database proof and final summary.

3:35-4:05 - DBeaver Database Check

Now I will switch to DBeaver. As you can see, this query joins `separation_job`, `uploaded_audio`, and `separation_result`. The latest row is at the top. It shows the job ID, audio ID, uploaded file name, status, output paths, timestamps, and processing time. If separation succeeds, the heart and lung paths are saved. If inference fails, the failed status is still saved, so the system keeps history and traceability.

4:05-5:00 - Final Summary

To summarize, as you can see, Facade makes the separation workflow simpler because `SeparationService` hides the subsystem details. Factory Method separates upload validator creation from the upload route. Strategy separates algorithm execution from the engine because `SeparationEngine` uses the `SeparationAlgorithm` interface and the current real strategy is `NeoSSNetStrategy`. The DBeaver query proves that the workflow is stored in SQLite. Overall, this supports SOLID because responsibilities are separated, route logic stays thin, and future validators or algorithms can be added with less impact.

Emergency reminder

If time is short, stop at only three lines: Facade at `separation_service.py` line 135, Factory Method at `upload.py` line 28, and Strategy at `separation_engine.py` line 29. Do not step deep into `NeoSSNet`. Stop at the design boundary and explain what the pattern proves.